

Graph Algorithms: Valid Path

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 30, 2026

© 2026 Arpit Raj. All rights reserved. Code and algorithms formatted for reference.

BFS (Valid Path)

Convert edge list → adjacency list.

Start BFS from source.

Mark source as visited and push into queue.

Pop node, if curr == destination → return true.

Push all unvisited neighbours.

If BFS ends without finding destination → false.

TC: $O(V + E)$

SC: $O(V)$

C++ Implementation

```
class Solution {
public:
    bool validPath(int n, vector<vector<int>>& edges, int source, int destination)
    {
        vector<vector<int>> adj(n);
        for(auto &it : edges){
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<int> seen(n, 0);
        queue<int> q;

        q.push(source);
        seen[source] = 1;

        while(!q.empty()){
            int curr = q.front();
            q.pop();

            if(curr == destination) return true;

            for(auto &newn : adj[curr]){
                if(!seen[newn]){
                    seen[newn] = 1;
                    q.push(newn);
                }
            }
        }
        return false;
    }
};
```

DFS (Valid Path)

Convert edge list → adjacency list.

DFS from source.

Mark current node visited.

If $\text{curr} == \text{destination} \rightarrow \text{return true}$.

Recursively visit unvisited neighbours.

If any recursive call returns true, propagate it upward.

If no path found, return false.

TC: $O(V + E)$

SC: $O(V)$ (visited + recursion stack)

C++ Implementation

```
class Solution {
public:
    vector<int> seen;
    vector<vector<int>> adj;

    bool dfs(int n, int curr, int destination){
        seen[curr] = 1;
        if(curr == destination) return true;

        for(auto &it : adj[curr]){
            if(!seen[it]){
                if(dfs(n, it, destination)){
                    return true;
                }
            }
        }
        return false;
    }

    bool validPath(int n, vector<vector<int>>& edges, int source, int destination) {
        adj.assign(n, vector<int>());
        for(auto &it : edges){
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        seen.assign(n, 0);
        return dfs(n, source, destination);
    }
};
```